



Syll: An Open-Source Multimodal Agent Harness for Teachable Personal Automation

Bo Zhang* Borui Zhang*† Chenghao Jiang* Minglei Shi Jie Zhou Jiwen Lu#

Tsinghua University, China

*Equal contribution. †Project Leader. #Corresponding author.

Project page: <https://github.com/THU-SAGE/syll>

Abstract. Personal AI agents increasingly need to operate across APIs, shells, web surfaces, and desktop GUIs, yet many systems remain optimized for a narrow interface or provide limited support for user teaching and audit. We present **Syll**, an open-source, self-hosted multimodal agent harness that unifies MCP/API tools, CLI execution, and visual GUI control within a modular runtime, aiming to expand agents’ ability to coordinate computer use across heterogeneous interfaces while streamlining the exchange of information between users and agents. At the center of Syll is a bidirectional user-agent thought converter: users can teach procedures through direct demonstration, which Syll converts into reusable skills, while agent execution is translated back into multimodal evidence such as logs, keyframes, and approval checkpoints. The system also externalizes memory, skills, routines, and governance as editable local artifacts, making inspection, extension, and secondary development straightforward. We conduct mechanism-oriented studies to validate the effectiveness of Syll’s multimodal routing, teachable GUI replay, and persistent local artifacts. We hope Syll can serve as a practical open-source foundation for personal automation systems that users can teach, inspect, and continuously extend.

1 Introduction

Personal AI agents are increasingly asked to complete real tasks rather than answer isolated questions. A single request may require retrieving local materials, running a script, checking a web dashboard, and operating a closed-source desktop application whose state is visible only on screen. In such settings, strong language modeling is necessary but insufficient. Reliable task completion depends on the surrounding **agent harness**: the layer that constructs context, selects interfaces, coordinates execution, and returns results in a form that users can review. Recent evidence suggests that harness design and agent-computer interface choice materially affect downstream performance [10, 16]. This motivates a systems perspective on personal automation.

A first unresolved question is how one agent should act across heterogeneous work surfaces. **MCP/API-style tools** are reliable when clean machine interfaces already exist [9, 13]. **CLI** execution is powerful for composable local control and long-horizon deterministic workflows [16]. Yet many practical tasks still depend on **GUI** interaction because state is only visible in pixels, APIs are unavailable, or users require screen-level evidence [12, 15, 19, 20]. Existing systems often privilege one interface family, forcing an avoidable trade-off between reliability and coverage. For personal automation, however, these surfaces are complementary: the right system should route each step through the narrowest interface that can complete the task while preserving the required audit surface.

A second question is how knowledge should flow between users and agents. Many current systems assume that operational knowledge must first be rewritten as prompts, schemas, or skill files before the agent can use it. This assumption is too restrictive. Many users know *how* to perform a task, but do not know how to formalize that procedure into a reusable representation. Conversely, when an agent completes a long-horizon task, a raw text trace is often too

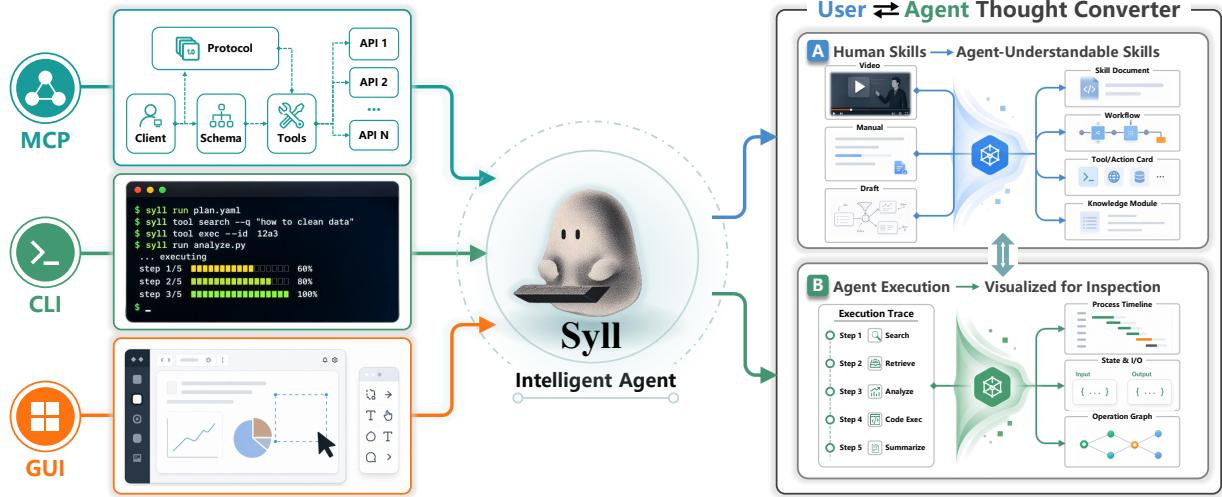


Figure 1. Syll overview. The system unifies three execution surfaces—MCP, CLI, and GUI—around a central multimodal agent harness, while the right branch visualizes the bidirectional user-agent thought converter that turns demonstrations into reusable skills and execution traces into reviewable multimodal evidence.

costly to inspect. We therefore argue that personal automation needs a **bidirectional user-agent thought converter**: human know-how should be converted into reusable machine-executable skills, and agent execution should be converted back into multimodal evidence such as screenshots, keyframes, logs, diffs, previews, and approval checkpoints.

We instantiate this view in **Syll**, an **open-source**, self-hosted multimodal agent harness for **teachable personal automation** shown in [Figure 1](#). Syll unifies MCP/API tools, CLI execution, and visual GUI control within one modular runtime. At its core, users can teach procedures through direct demonstration, which Syll converts into reusable skills, while the runtime translates execution back into multimodal evidence for efficient review. Beyond execution, Syll externalizes memory, skills, routines, governance, and traces as **persistent local artifacts**, making the system inspectable for users and straightforward to extend for developers.

This design is motivated by both usability and research value. Keeping execution surfaces, skill conversion, evidence generation, channels, context construction, and confirmation gates explicit reduces cross-component entanglement and lowers the barrier to secondary development. It also improves auditability: as agent task horizons continue to grow [7], and as open-ended computer-use settings continue to expose substantial verification burden [1], practical usefulness depends not only on model intelligence but also on whether the system is **teachable**, **auditable**, and **continuously extensible**. We therefore evaluate Syll through mechanism-oriented studies aligned with these goals.

This technical report makes the following contributions:

- **Open-source multimodal agent.** We present Syll as an open-source multimodal agent harness that supports MCP/API, CLI, and GUI as complementary execution surfaces and routes tasks to the most appropriate action layer.
- **A bidirectional user-agent thought converter.** We design a teachable workflow that turns direct user demonstrations into reusable skills and an evidence pipeline that translates agent execution back into reviewable outputs.
- **Persistent local artifacts and modular design.** We externalize memory, skills, routines, traces, evidence, and governance as user-editable local artifacts within a modular architecture, making the system extensible.
- **Mechanism-oriented evaluation.** We validate Syll through studies targeted at three core system properties: multimodal routing, teachable GUI replay, and persistent local artifacts.

Table 1. High-level comparison of agent system families. “Partial” means the capability exists but is not the primary design center.

System family	GUI	CLI/API	Demonstration	User memory	Main audience
CLI-only coding agents	No	Yes	No	Partial	developers
MCP/tool frameworks	No	Yes	No	Partial	developers/integrators
GUI agents	Yes	Partial	Partial	Partial	automation builders
Record-replay/RPA tools	Yes	Partial	Yes	No	operations teams
Companion chat agents	No	Partial	No	Yes	end users
Syll	Yes	Yes	Yes	Yes	users and developers

2 Related Work

2.1 Agent-Centric Task Execution

Recent progress in autonomous agents has concentrated on strengthening reasoning, planning, and perception. Foundational frameworks such as ReAct [17] and Reflexion [14] introduced iterative reasoning-action loops that allow language models to decompose tasks, interact with environments, and refine behavior through textual feedback. Subsequent work extended these paradigms to tool invocation and long-horizon execution, enabling agents to call external APIs and structured functions [13, 16]. With the emergence of vision-language models, the interaction surface has broadened to graphical user interfaces. Systems like Claude Computer Use [2], Operator [10], and UI-TARS [12] perceive and act upon GUIs through screenshots and low-level control primitives. Despite these advances, agent-centric approaches typically treat the execution surface as a fixed given, an API, a browser, or a specific GUI environment, and concentrate on what the agent decides to do. Consequently, interaction knowledge, such as which pixels to click, which states to wait for, and what outcomes to expect, remains confined within a single trajectory transcript and is rarely externalized as a reusable, auditable artifact that can be repurposed by the user or the system.

2.2 Harness-Centric and Record-and-Replay Systems

Complementing the advances in agent intelligence, a line of work targets the system infrastructure that supports reliable, locally owned agent execution. Frameworks such as OpenClaw [11] and NanoBot [4] provide local-first runtimes that integrate persistent memory, tool execution, scheduling, and multi-channel communication. The Model Context Protocol (MCP) [9] standardizes how agents interact with external tools through structured, schema-driven APIs. These harness-centric systems offer strong modularity and extensibility but are predominantly confined to textual or structured modalities; GUI interaction, when present, is often treated as a separate subsystem rather than a unified modality.

Record-and-replay tools and programming-by-demonstration (PbD) systems address a complementary need: enabling users to automate repetitive GUI workflows without scripting. Sikuli [18] uses screenshot-driven search to replay visual procedures, CoScripter [8] captures step-by-step action descriptions, and Eager [3] proactively detects iterative patterns. More recent efforts like ShowUI-Aloha [19] incorporate human-taught GUI trajectories for agent learning. These approaches excel at capturing what the user does, but they typically lack a semantic phase structure, a persistent skill registry shared with non-GUI tools, and an evidence pipeline that translates execution back into inspectable artifacts. As a result, a recorded workflow often remains a brittle, isolated macro rather than a first-class runtime abstraction that can be retrieved, scheduled, verified, and refined alongside other skills.

2.3 Teachable Automation and Execution Audit

Two design goals that are central to Syll, allowing users to *teach* procedures through demonstration and providing *audit evidence* for every executed action, remain underrepresented in the current agent landscape. On the teaching side, existing PbD and record-and-replay tools show that end-user demonstration is a viable way to transfer operational knowledge, but they do not connect this knowledge to a general-purpose agent’s context construction or to a shared skill registry that also includes MCP tools and CLI commands. On the audit side, agent evaluation has largely been success-rate driven; benchmarks like OSWorld [15], WebArena [20], and OSWorld-Human [1] measure task completion, yet few systems explicitly structure execution traces into layer-specific, user-reviewable evidence artifacts such as keyframes, diffs, semantic expectations, and approval records. Syll’s bidirectional thought converter treats teaching and auditing as two sides of the same artifact lifecycle: a user demonstration is packaged as an inspectable skill with embedded

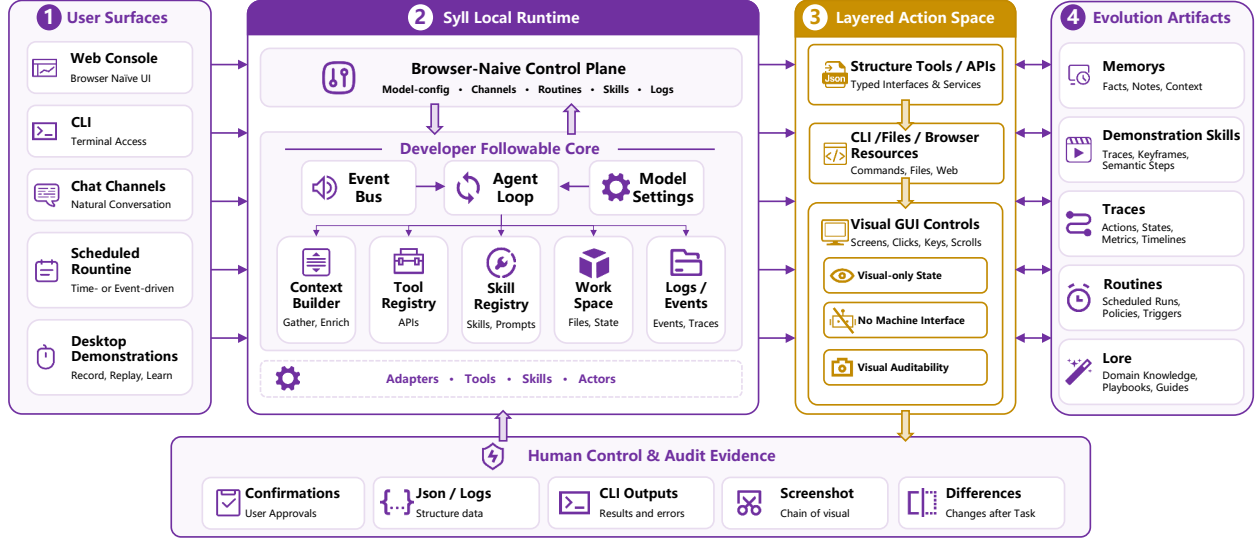


Figure 2. Syll runtime overview. Requests enter one local runtime, are grounded in shared workspace context, routed through tools, commands, or GUI control, and leave audit evidence as confirmations, logs, screenshots, keyframes, and workspace diffs.

visual cues and post-state expectations, while runtime execution—whether over MCP, CLI, or GUI—generates a corresponding audit trail that satisfies a minimum coverage constraint, making the agent’s observation-decision-action loop of the agent transparent long after the task finishes.

2.4 From Fragmented Capabilities to a Unified Artifact Lifecycle

Taken together, prior work reveals a structural gap: no existing system jointly resolves multi-surface execution, user-teaching workflows, and persistent, user-inspectable execution evidence within a single framework. Table 1 summarizes this landscape qualitatively: GUI agents provide visual control but lack demonstration handling; MCP/tool frameworks offer structured APIs but no GUI or teaching primitives; record-replay tools support demonstration but omit cross-surface routing and audit; companion chat agents emphasize user memory but are not designed for GUI automation or audit trails. Syll addresses this gap by elevating execution surfaces, skill registration, evidence generation, and memory artifacts into a unified, modular runtime. The resulting system treats computer-use as an artifact lifecycle, from demonstration to skill, from execution to audit trail, and externalizes all mutable state as editable local documents, enabling users to teach, inspect, and continuously extend their personal automation.

3 Method

Syll is an open-source, self-hosted computer-use harness for multimodal personal automation. It is designed for the practical setting in which a user moves between a phone, a browser, a terminal, local files, scheduled routines, and desktop applications, while expecting one local agent to preserve context and act across these surfaces. Syll implements a continuous lifecycle that captures user intent across everyday interfaces, routes actions through the most appropriate machine or visual modality, accumulates context as editable memory artifacts, and distills successful workflows into reusable capabilities.

3.1 System Positioning and Mechanism Claims

Syll targets recurring personal workflows that span heterogeneous surfaces, such as initiating a task in chat, manipulating local files, and confirming actions on mobile. While single-interface agents struggle with this fragmentation, Syll treats diverse interfaces as a complementary ecosystem. Structured tools (e.g., MCP) ensure reliability for API-ready applications, CLI operations provide deterministic control, and visual GUI actions serve as a critical completeness layer when APIs are absent or screen-level evidence is required.

Syll is organized around four mechanism claims that operationalize the system contributions of Section 1: C1 instantiates

Table 2. Core notation for the typed artifact-option process, grouped by runtime role.

I. State and channel input			
$S_t = (X_t, A_t)$	joint runtime state	$\xi_t = (c_t, e_t)$	channel event (channel, raw)
X_t	external computer state	$O(\cdot \mid S_t)$	observation kernel
A_t	agent-owned artifacts	$m_t = \text{Adapter}_{c_t}(e_t)$	normalized message
II. Typed context projection			
$C_t = \Phi(A_t, m_t)$ with the components below.			
W_t	editable workspace	T_t	tool/schema set
H_t	session history	V_t	media and visual observations
R_t	skill registry	P_t	schedules, policies, confirmations
III. Policy, typed action, and execution			
$\pi_\theta^R, \pi_\theta^A$	route and action policies	Exec_{r_t}	route-specific executor
r_t, α_t	execution route, typed action	o_{t+1}, z_{t+1}	observation, evidence artifact
$\hat{\psi}_t, \chi_t$	expected post-state, confirmation gate	U_A	artifact-state update
$a_t = (r_t, \alpha_t, \hat{\psi}_t, \chi_t)$ —full step record (route, typed action, expected post-state, confirmation gate).			
IV. Skill option and auditability constraint			
$s = (\Omega_s, \pi_s, \beta_s, \Psi_s)$	skill option (init, policy, term, post)	Ψ_s^{terminal}	terminal postcondition
q_t, D_s	phase belief, demonstration set	$\text{Coverage}(z_{1:T}, \Psi) \geq \kappa$	audit coverage constraint

the multimodal agent contribution, C2 and C3 jointly instantiate the bidirectional user-agent thought converter, and C4 instantiates persistent local artifacts and modular design.

C1: multimodal work surfaces. A personal agent must operate seamlessly across the diverse interfaces where users naturally work, including web consoles, CLIs, chat channels, and desktop GUIs. To prevent context fragmentation, these integrated surfaces must share a unified workspace, event stream, skill registry, and model configuration.

C2: user demonstrations to skills. Users should be able to impart operational knowledge by demonstrating real-world workflows, bypassing the need for rigid text prompts or schemas. This demonstration-driven approach bridges human practice and executable skills by capturing essential visual cues, phase structures, and expected post-action states for reliable replay.

C3: multimodal evidence generation. Long-horizon task execution must yield reviewable evidence rather than an opaque text transcript. To achieve this, Syll records comprehensive route-specific artifacts, including tool outputs, command logs, workspace diffs, screenshots, keyframes, semantic traces, and approval records. This ensures users can transparently inspect what the agent observed, decided, and altered during execution.

C4: modular system design. Memory, skills, routines, traces, evidence, and governance are structured as explicit modules backed by user-owned local artifacts. This architectural modularity lets users directly inspect and modify the system state, while also letting developers extend channels, tools, actors, schedules, and skill types without altering the underlying core loop.

Typed artifact-option process. Before introducing the process equations, Table 2 summarizes this subsection’s core notation. Because the runtime-verification audit interface is still being finalized, this formulation focuses on the stable state-action-evidence abstraction exposed by the current runtime.¹

We model Syll as a typed artifact-option process rather than a single reasoning-action transcript. The joint state contains both the external computer state and the agent-owned artifact state:

$$S_t = (X_t, A_t).$$

A channel event is a partial observation of this state. Writing $\xi_t = (c_t, e_t)$ for the observed channel and raw event,

$$\xi_t \sim O(\cdot \mid S_t), \quad m_t = \text{Adapter}_{c_t}(e_t).$$

¹Implementation details of the runtime-verification audit interface are in progress and will be documented in a future release.

The context builder then constructs a typed, task-relevant projection

$$C_t = \Phi(A_t, m_t) = (W_t, H_t, R_t, T_t, V_t, P_t),$$

whose components are listed in [Table 2](#). Because the true computer state is partially observable and the prompt is finite, C_t is a lossy projection of the belief, not a sufficient statistic. The policy first chooses an execution route and then emits a typed action:

$$r_t \sim \pi_{\theta}^R(\cdot \mid C_t), \quad (\alpha_t, \hat{\psi}_t, \chi_t) \sim \pi_{\theta}^A(\cdot \mid C_t, r_t),$$

with $a_t = (r_t, \alpha_t, \hat{\psi}_t, \chi_t)$. Here χ_t denotes the confirmation requirement emitted with the action, such as no gate, preview required, or explicit user approval; it is not a model confidence score and is retained as part of the audit record. Execution returns both an observation and an evidence artifact, after which the artifact state is updated:

$$(o_{t+1}, z_{t+1}) = \text{Exec}_{r_t}(\alpha_t, X_t), \quad A_{t+1} = U_A(A_t, m_t, a_t, o_{t+1}, z_{t+1}, \hat{\psi}_t).$$

A reusable skill is a temporally extended action,

$$s = (\Omega_s, \pi_s, \beta_s, \Psi_s),$$

where π_s marginalizes over a phase belief q_t defined in [Section C](#) and β_s is derived from the terminal postcondition Ψ_s^{terminal} . We characterize ReAct as a degenerate case of this process—single channel, A_t collapsed to a textual transcript H_t , no route selection, no option layer, and policy output $(\tau_t, \alpha_t) \sim \pi_{\theta}(\cdot \mid H_t)$ over a textual rationale τ_t —and full skill replay as a Bayesian phase belief over D_s in [Section C](#). Auditability enters the optimization as a coverage constraint $\text{Coverage}(z_{1:T}, \Psi) \geq \kappa$ rather than a soft reward, formalized in [Section C.1](#). The remainder of this section details how the runtime, action space, skill system, evidence layer, and document workspace implement C1–C4.

3.2 Cross-Surface Runtime and Browser-Native Control

Cross-surface continuity begins at the runtime. Every user-facing surface enters one local event loop and shares the same workspace context, so the agent operates over a single ongoing state rather than a set of disjoint applications. Syll realizes this through a unified channel-bus-agent-tool architecture ([Figure 2](#)). Channel adapters normalize inputs from diverse sources (e.g., web consoles, CLIs, messaging apps, and scheduled jobs) into a standardized event format. An in-process event bus then coordinates these inbound messages, proactive triggers, and tool observations. Regardless of the origin surface, the core agent loop processes all events uniformly: it loads the relevant session, constructs the context C_t , queries the model, executes validated tool calls, and appends the resulting observations to the session history.

The browser-native control surface is integral to the system method. Through the web UI, users can directly inspect and edit core artifacts, including model configurations, identity documents, skills, and activity logs. Unlike developer-oriented gateways such as OpenClaw [\[11\]](#), Syll treats browser-based interaction as a first-class runtime surface. This design lowers the barrier to entry, rendering the agent’s internal state transparent and easily manipulable for non-expert users.

Beyond serving end users, this architecture is explicitly designed for research and community extension. Syll decouples its core loop from peripheral interfaces, ensuring that channels, tools, and skills interact via standardized boundaries and plain-text persistence rather than tightly coupled code. This strict modularity establishes the runtime as a transparent foundation, empowering researchers to effortlessly isolate components, integrate new capabilities, and fork the system for custom experiments.

3.3 Complete Action Space: Tools, Commands, and Visual GUI Control

Where the runtime unifies inputs, the action space unifies outputs. Rather than forcing all tasks into a single interface paradigm, Syll dynamically routes each request through structured tools, local commands, or visual GUI control, organized as a progressive fallback that trades off execution precision against universal applicability across three layers.

Structured API Layer: This primary route comprises schema-driven tools and standardized connectors (e.g., MCP). It is prioritized whenever an application exposes a clean machine interface, as function calling guarantees deterministic execution and explicit input-output contracts.

Command-Line Layer: Serving as the first fallback, this layer encompasses shell operations, local scripts, and filesystem interactions. When APIs are absent but the environment state remains textually accessible, the CLI provides a composable, reproducible substrate for chaining local operations.

Visual GUI Layer: This final layer acts as the completeness fallback, extending the agent’s reach to all standard software. If the state is encoded exclusively in pixels or if users demand screen-level audit evidence, Syll reasons over screenshots and dispatches low-level desktop events. This ensures legacy applications and human-facing workflows remain accessible without requiring custom connectors.

Table 3. Action-layer routing policy. Syll selects the narrowest route that can complete the task while preserving the audit surface required by policy or by the user.

Route	Selected when	Evidence retained
Structured tools/API	A schema-described operation, connector, file tool, delivery tool, or configuration route exists	Tool name, validated arguments, output/error, confirmation record
Local command/resource	Desired state is machine-readable or reproducible through shell, scripts, local files, or browser-accessible resources	Command, working directory, stdout/stderr, generated files or diffs
Visual GUI control	State is only visible in pixels, no reliable API exists, or the user needs screen-level evidence	Screenshots, keyframes, low-level actions, visual observations, semantic expectations
Confirmation gate	External delivery, destructive local change, account change, purchase, or scheduled send would create a user-visible side effect	Proposal, candidate artifact, approval token, final side-effect record

Furthermore, this layered architecture ensures that audit evidence remains strictly layer-specific. A **structured API** call yields validated arguments and machine-readable results; **command-line control** produces reproducible commands and textual outputs; and **visual GUI control** retains screenshots, keyframes, and action traces. Consequently, Syll routes actions based not only on operational feasibility but also on the specific evidence required to establish user trust.

3.4 Demonstration-Teachable Skills and Replay

Skill abstraction and registration. Some operational knowledge is easier to demonstrate than to describe. Task phases, visual cues, wait conditions, and expected post-action states all live more naturally in a recorded demonstration than in a written prompt, and capturing them is what makes the resulting skill different from a coordinate replay. Syll stores each desktop demonstration as a reusable runtime artifact, preserving a natural recording interface for users while exposing a state-aware replay representation to the agent through the same unified registry used for text-based skills.

Building on programming-by-demonstration and demonstration-guided GUI automation [3, 8, 18, 19], our contribution lies in elevating the demonstration to a first-class runtime abstraction. Each skill is encapsulated as a self-contained directory with a well-defined schema (Table 4) that connects the recording, replay, and audit subsystems. Once published, this abstraction is registered into the agent context $R \subset C_t$, accessible via the web UI, invocable through natural language, and schedulable as a proactive routine.

Table 4. Demonstration-skill schema. Each field has a concrete replay role; N is the number of semantic steps in the demonstration.

Field	Stored content	Replay and audit use
<i>meta</i>	Name, app context, aliases, preferred replay mode, terminal <code>success_check</code> predicate	Registry summary in R ; channel dispatch; terminal check in Algorithm 1
<i>traj</i>	Ordered raw events: <code>click</code> , <code>type</code> , <code>key</code> , <code>scroll</code> , <code>wait</code> , <code>hotkey</code>	Desktop executor input; deterministic fallback replay
<i>kf</i> [i]	One screenshot per semantic step, $i \in [0, N-1]$	Action ICL image anchor; human-readable audit evidence
<i>trace</i> [i]	(<code>obs_i</code> , <code>intent_i</code> , <code>action_i</code> , <code>expectation_i</code>), where <code>expectation_i</code> is a post-state predicate	Phase indexing; actor prompt construction; per-step verification

The implementation tuple $s = (meta, traj, kf, trace)$ is the artifact form of the option $s = (\Omega_s, \pi_s, \beta_s, \Psi_s)$ defined

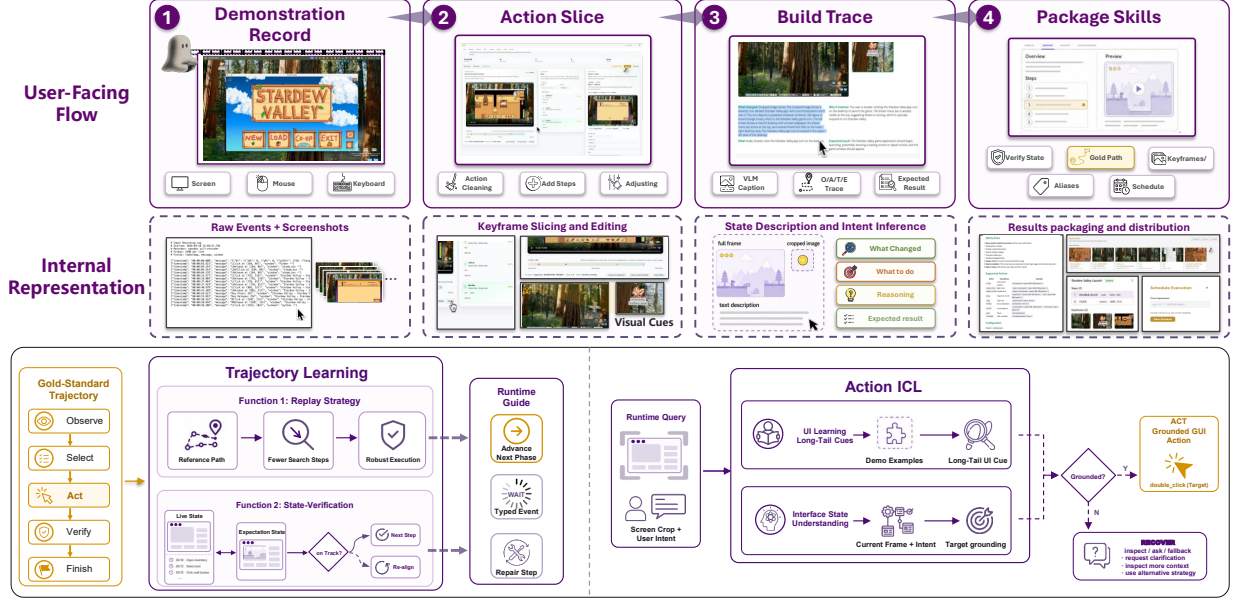


Figure 3. Demonstration-conditioned replay in Syll. The user-facing flow records a desktop routine, slices it into editable actions, builds semantic traces, and packages the result as a reusable skill; the internal representation keeps raw events, keyframes, state descriptions, expected results, aliases, schedules, and audit evidence. During replay, trajectory learning supplies a reference path for efficient execution and self-verification, while Action ICL grounds local GUI actions by learning long-tail UI cues and understanding the current interface state.

in Section 3.1; the integer phase index used by Algorithm 1 is the MAP collapse of the Bayesian phase belief $q_t(i)$ formalized in Section C.

Figure 3 separates skill construction from replay-time use. The replay loop operates through two mechanisms: phase indexing and in-context action grounding.

Phase indexing. Replay maintains a phase index $p \in [0, N]$, initialized at 0, that tracks the active step in the demonstration. This operationalizes *trajectory learning*, giving the actor a persistent task-state signal beyond the current screenshot.

Action ICL. At step k with current screenshot I_k and phase p , the actor assembles an in-context window $E_k = \{(kf[i], trace[i]) : i \in [\max(0, p-1), \min(N-1, p+1)]\}$ spanning the previous, current, and next demonstrated steps. Retrieval relies on phase position p rather than learned visual similarity, since p is the task-state signal that a similarity retriever would otherwise need to infer. This choice keeps replay deterministic and maintains an auditable link to the demonstrated trajectory. Conditioning the next-action decision on E_k provides explicit grounding for long-tail UI cues such as localized labels, repeated controls, and visually similar icons that are ambiguous from I_k alone.

Phase update. After the executor performs an action, the actor captures I_{k+1} and evaluates three predicates, the current expectation $expectation_p$, the next expectation $expectation_{p+1}$, and the terminal success_check. Predicate evaluation is performed as a short structured judgment over the screenshot. Replay terminates on success_check, advances $p \leftarrow p+1$ when the current expectation is satisfied, also advances when only the next expectation is satisfied, and otherwise holds p for recovery.

Wait and recovery. WAIT is emitted when the current screenshot is a transient pre-state of the active expectation, such as a loading screen, animation, or application launch. It is recorded as a typed executor event, so the audit log and action budget distinguish waiting from task progress. RECOVER re-grounds against E_k without advancing p when no expected predicate matches. Together, these affordances allow replay to preserve task state across asynchronous interface transitions, where a screenshot-only policy would otherwise repeat completed phases.

Algorithm 1 integrates the schema, retrieval rule, phase update, and success check into a single replay episode. All replay paths dispatch through the shared desktop executor and emit fresh keyframes, semantic traces, and confirmation events, ensuring that replayed runs share the same artifact shape as original demonstrations and remain uniformly

auditable.

Algorithm 1 Demonstration-conditioned replay (one episode).

Require: Skill $s = (meta, traj, kf, trace)$ with N semantic steps; instruction u ; context C_t ; step budget K ; wait interval δ

```

1:  $p \leftarrow 0$ ; evidence  $E \leftarrow \emptyset$  {phase index, audit log}
2: for  $k = 1$  to  $K$  do
3:    $I_k \leftarrow \text{CaptureScreenshot}()$ ;  $E \leftarrow E \cup \{I_k\}$ 
4:   if  $\text{SuccessCheck}(I_k, meta.success\_check)$  then
5:     return  $\langle \text{SUCCESS}, E \rangle$ 
6:   end if
7:    $W \leftarrow [\max(0, p-1), \min(N-1, p+1)]$  {phase-positional window}
8:    $E_k \leftarrow \{(kf[i], trace[i]) : i \in W\}$  {Action ICL pack}
9:    $r \leftarrow \text{Actor}(C_t, u, E_k, p, I_k)$  { $r.kind \in \{\text{ACT}, \text{WAIT}, \text{RECOVER}\}$ }
10:  if  $r.kind = \text{WAIT}$  then
11:     $\text{Sleep}(\delta)$ ;  $E \leftarrow E \cup \{r\}$ ; continue
12:  end if
13:  if  $r.kind = \text{RECOVER}$  then
14:     $E \leftarrow E \cup \{r\}$ ; continue {re-ground next iteration;  $p$  unchanged}
15:  end if
16:   $\text{DesktopExecutor}(r.action)$ ;  $E \leftarrow E \cup \{r.action\}$ 
17:   $I_{k+1} \leftarrow \text{CaptureScreenshot}()$ 
18:  if  $\text{Match}(I_{k+1}, trace[p].expectation)$  or  $(p+1 < N \text{ and } \text{Match}(I_{k+1}, trace[p+1].expectation))$  then
19:     $p \leftarrow p + 1$  {advance, possibly through a transient state}
20:  end if
21: end for
22: return  $\langle \text{BOUNDED-FAILURE}, E \rangle$ 

```

This design distinguishes Syll from screenshot-only visual agents. Instead of inferring task structure from the current frame at every step, Syll couples local action grounding with temporal state maintenance. Keyframes and semantic traces condition the current GUI action through Action ICL, while the phase index, historical trajectory, and next-state expectations preserve progress through the demonstrated routine. The resulting executable artifact remains reusable, inspectable, schedulable, and sharable.

3.5 Document-Mediated Bidirectional Self-Evolution

Self-evolution in Syll happens at the artifact layer rather than the weight layer. Identity, operating constraints, user memory, narrative context, and proactive routines all live as plain editable files, refined continuously by the user and by Syll’s own writes. This design keeps the agent’s growth transparent and directly under user control, in contrast to opaque profile services or fine-tuning loops.

Syll operationalizes bidirectional self-evolution through a user-owned document workspace. Instead of sequestering companion state within a hidden profile, the runtime externalizes identity, operating constraints, user memory, narrative context, and proactive routines as editable local documents. During context construction, the builder resolves workspace variables and injects the relevant document slices into C_t . Consequently, any modification to the workspace—whether by the user or the agent—immediately modulates subsequent behavior.

This workspace serves as the primary interface for co-evolution between the user and the agent (Figure 4). Users can refine identity boundaries, communication tone, and behavioral rituals directly in plain text. Simultaneously, Syll updates local memory artifacts based on conversations, tool outcomes, and user feedback. The resulting state is not an opaque memory service but a transparent, versionable layer that users can backup, audit, or delete at will.

The name *Syll* encapsulates this architectural stance. Much like a syllable whose meaning is derived from the surrounding utterance, the agent begins with a structured but incomplete identity, gaining specificity through accumulated documents. Profile documents situate the user, lore documents preserve relationship context, and routine documents specify proactive boundaries. Persona thus becomes an ambient, document-mediated interface for long-term interaction, remaining both persistent and fully user-governed.

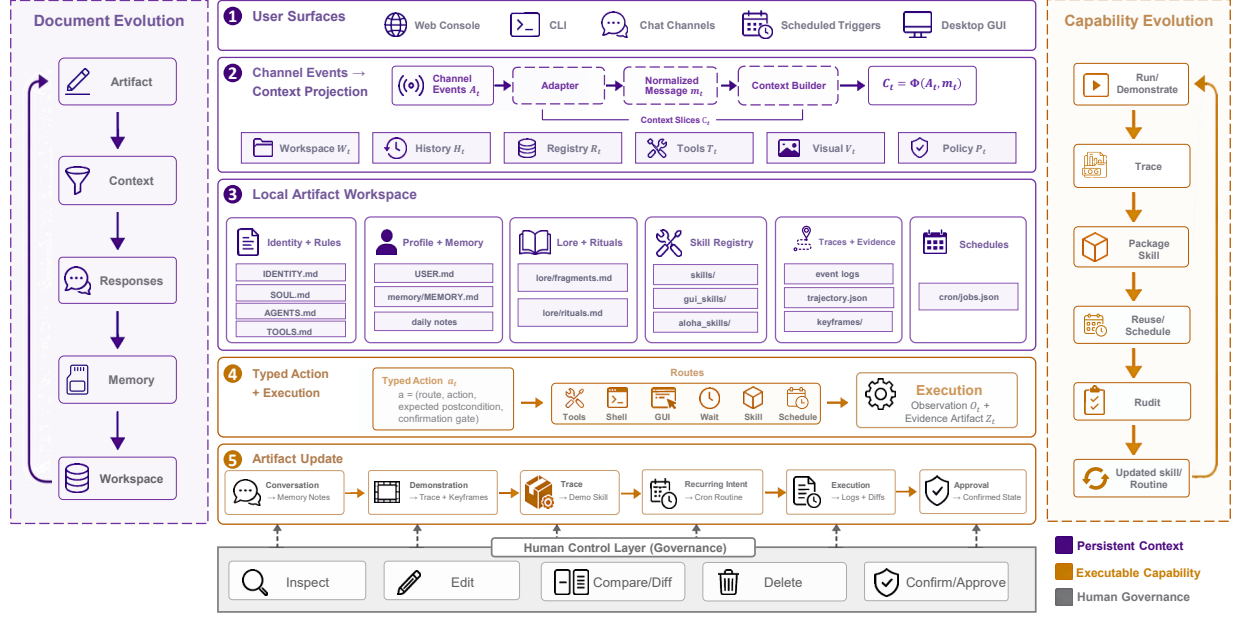


Figure 4. Artifact-mediated self-evolution. Syll evolves through editable local artifacts rather than model-weight updates. The document loop lets the user and Syll co-evolve through identity, rules, profile, lore, and routine markdown files. The capability loop turns runs and demonstrations into traces, skills, scheduled routines, and refinement evidence, while human control stays explicit through inspection, editing, diffing, deletion, and approval.

3.6 Capability Evolution through Traces, Skills, and Routines

While Section 3.5 addressed companion-state evolution, this subsection focuses on capability evolution (Figure 4). Capability evolution is organized around three interlocking artifact families. Traces preserve execution experience, skills distill repeatable procedures, and routines elevate recurring needs into proactive opportunities. Together, they convert runtime interactions into a local capability substrate that the agent can retrieve, execute, inspect, and refine.

Traces record the procedural and episodic structure of each execution. For GUI tasks, a trace stores screenshots, keyframes, low-level actions, semantic expectations, and final observations. Conversely, for tool and shell operations, it captures tool names, arguments, outputs, errors, and user confirmations. Rather than serving merely as static logs, traces constitute a structured experience record that facilitates failure diagnosis, replay comparison, and downstream skill authoring.

Skills consolidate repeatable procedures into declarative, addressable capabilities. Markdown skills express textual or tool-oriented procedures, demonstration skills package state-aware visual workflows, and browser-managed skills surface the same artifacts through the runtime registry. All skill types share this registry abstraction, allowing the agent to discover them through context C_t and the user to inspect, edit, version, or share them as standalone artifacts.

Routines elevate recurring needs into scheduled, proactive behaviors. Time-based cron jobs and event-driven rituals trigger the core agent loop using a specialized prompt, enabling the agent to deliver messages, invoke skills, or remain silent based on the context. Notably, silence is treated as a valid contextual decision rather than a failure mode, ensuring the proactive system acts only when genuinely necessary. Over time, routines inextricably bind the user’s habits to the document workspace and skill registry, transforming recurring patterns into auditable, time-aware automation.

3.7 Multimodal Evidence, Human Control, and Open-Source Extensibility

Per-layer evidence and user-owned artifacts jointly form Syll’s trust boundary, translating transient execution states into a persistent substrate that the user can read, audit, or revise. The resulting governance model rests on three pillars. **Evidence** provides retrospective transparency, **approval** provides proactive safety, and **ownership** provides long-term data sovereignty.

First, actions produce layer-specific audit evidence at every level of the action space. Structured API calls log validated

arguments and machine-readable results, command-line operations preserve commands, working directories, outputs, and generated diffs, and GUI execution retains screenshots, keyframes, action records, and semantic traces. Together, these artifacts allow users to reconstruct what the agent observed, decided, and executed, regardless of which layer carried out the action (Figures 2 and 4).

Second, side-effectful operations are separated into proposal and execution. For example, file retrieval can search local files, generate previews, present candidate paths, and wait for explicit user selection before attaching or sending a file. The same confirmation pattern applies to external delivery, destructive local changes, account changes, purchases, and scheduled sends. Syll therefore treats user-visible consequences as approval-gated actions rather than direct model outputs.

Third, Syll keeps both state and extensibility open. Companion state is stored as plain local artifacts that users can inspect, edit, fork, or delete, while channels, tools, actors, schedules, and skill types are exposed through explicit extension boundaries. This lets users own the agent’s state and lets researchers extend the runtime without rewriting the core loop.

Together, these three principles position Syll as an open computer-use environment rather than a closed assistant product. The agent acts across a broad multimodal action space, but its automation behavior is anchored in artifacts the user can read, audit, or revise.

4 Evaluation

Our evaluation targets the heterogeneous personal-automation setting that Syll’s method addresses, where requests cross chat surfaces, local files, command-line tools, browser sessions, desktop GUIs, and scheduled routines. We organize the study suite around three of the four mechanism claims of Section 3: Study A targets C1 (action-layer routing), Study B targets C2 (demonstrations to skills), and Study C targets C4 (artifact persistence); the fourth claim, C3 (multimodal evidence), is exercised transversally as the audit surface that every study reports. Across studies, the primary evidence comprises five fields: intent, route decision, execution trace, produced artifact, and audit surface. Timing, model calls, tokens, screenshots, and estimated cost are reported only as local efficiency metadata, not as cross-system rankings. Table 5 consolidates this map together with each study’s controlled contrast and main readout.

Table 5. Evaluation map. Each study isolates one mechanism and reports a compact auditable readout.

Study	Mechanism isolated	Controlled contrast	Main readout
Study A Action routing	layer choice across tools, CLI, and visual inspection	three probes: file delivery, local command, screenshot fallback	3/3 route matches; confirmations and audit artifacts retained
Study B Demo replay	effect of a recorded trajectory in clean GUI replay	demo-guided replay vs instruction-only replay	3/3 vs 2/3 phases; 0 vs 3 off-trace actions
Study C Artifact evolution	persistence, refinement, and routine promotion	Full Syll vs Frozen Workspace across five delayed episodes	5/5 + 4/4 reuse; refine=1 ; routine=1

The three studies are intentionally complementary: Study A and Study B test model-driven runtime behavior under controlled probes and share a model backbone, action executor, and evaluation harness, whereas Study C uses a deterministic artifact checker with scripted user turns so that workspace and routine state are isolated from model-side extraction quality. Aggregate run metadata for the two model-driven studies appear in Table 11.

4.1 Study A: Cross-Surface and Action-Space Coverage

Study A evaluates whether Syll routes through the appropriate action layer when the task itself does not specify an interface. We construct three probes that admit different interface choices: a side-effectful file delivery, a deterministic local command, and a screen-only visual inspection. A run is counted as passing when the selected layer matches the intended interface, the task-specific outcome anchor is satisfied, the corresponding audit evidence is retained, and no side-effect error occurs. Run-level timing, calls, tokens, and screenshots are reported alongside, but they do not enter the pass criterion.

A1: confirmation-first file retrieval. The user asks Syll to locate a local presentation from a chat surface and selects one candidate by visual attribute (“the green one”), where the result is delivered through a side-effectful channel. The probe tests cross-surface continuity, structured file tools, preview-based disambiguation, and the confirmation invariant for side-effectful delivery.

A2: deterministic local command. The user asks for a local computation, where the target state is machine-readable through a local command. The probe tests that Syll defaults to the local command layer rather than visual control when a more deterministic interface is available.

A3: visual-state fallback. The user asks for a screen-level inspection, where the relevant state is only available on screen. The probe tests the visual state-inspection layer in isolation, without invoking multi-step GUI manipulation.

Table 6. Study A result ledger. The main result is route selection and retained audit evidence; efficiency metadata are secondary.

Probe	Route	Outcome anchor	Evidence	Time	Calls	Tokens	Err.
A1 File	structured → confirm	selected and attached the green Aurora roadmap deck	previews, paths, confirmation	25.0 s	6	85.7 k	0
A2 CLI	local CLI	returned <code>cwd</code> and top-level entry count without GUI	command log, stdout	4.5 s	2	26.6 k	0
A3 Visual	visual screenshot	captured one screenshot and summarized visible UI state	screenshot, state summary	13.3 s	2	28.9 k	0
Suite	3/3 match	3/3 pass	2 confirm gates	42.8 s	10	141.1 k	0

The three runs each selected the expected layer and produced the corresponding audit evidence. A1 invoked structured file tools and previews, blocking delivery until the user supplied a confirmation token. A2 invoked the local command layer and returned machine-readable workspace state. A3 invoked the visual layer once and retained the screenshot as audit evidence. Together, the runs follow the layered routing policy in Table 3, selecting the narrowest viable layer for each probe.

4.2 Study B: Demonstration-Trajectory Ablation

Study B isolates the contribution of the recorded user demonstration. We fix the GUI actor, executor, task instruction, and initial interface setup, and change only whether Syll provides the demonstration skill artifact. This controlled difference lets the result speak directly to two method mechanisms: Action ICL grounding for local UI cues, and demonstration-conditioned phase indexing across asynchronous interface transitions. In the formal language of Section C, this corresponds to the phase belief q_t collapsing to its MAP estimate $p = \arg \max_i q_t(i)$ used by Algorithm 1. The task is a fixed Stardew Valley start workflow with three semantic phases. Its full specification, including phase targets, skill artifact, conditions, and evidence metrics, is given in Table 7.

Table 7. Study B protocol card. The controlled difference is whether the replay receives the demonstration skill artifact; the task, executor, actor, and initial GUI state are fixed.

Field	Value
Task	From the macOS desktop, launch Stardew Valley, open the Load menu, select the existing <code>Vlm/VLMBot</code> farm save (the in-game character names of the saved playthrough), and stop once loading starts or the farm world is visible
Phase targets	<code>launch_stardew_app</code> , <code>open_load_menu</code> , <code>load_vlm_save</code>
Skill artifact	<code>skills/stardew_start/trajectory.json</code> : keyframes, semantic trace, and expected post-action states
Conditions	demo-guided replay vs instruction-only replay
Evidence metrics	final state, phase coverage, off-trace actions, repeated phase visits, cue grounding, screenshots, calls, and wall time
Scope	clean replay on the same machine, actor, executor, and initial interface setup

The Stardew Valley start workflow is chosen because it jointly exercises three demonstration-relevant signals. Its UI contains long-tail cues such as a Load button, an hourglass save icon, and the player-and-farm save label, which probe Action ICL grounding. Its three semantic phases must occur in order with no shortcut, which probes demonstration-conditioned phase indexing. Its multiple loading and animation phases produce asynchronous transitions, which probe the typed WAIT/RECOVER loop. A terminating farm-world HUD provides a deterministic visual success condition.

We compare two replay conditions. In **demo-guided replay**, the actor receives the task instruction u , the current screenshot I_k , and the demonstration skill artifact (Table 4) consisting of keyframes k_f and semantic traces $trace$. In **instruction-only replay**, the actor receives only u and I_k . The two conditions differ only in the demonstration artifact, so any difference in phase coverage, off-trace actions, or cue grounding is attributable to it.

Table 8. Study B clean demonstration-trajectory ablation. Demonstration guidance mainly changes phase indexing and long-tail cue grounding, not the model-call budget.

Evidence item	Demo-guided replay	Instruction-only replay
Result	Pass ; farm HUD visible: Wed. 3, 6:00 am, 50g	Fail ; target save not loaded
Phase coverage	3/3 ; launch $\rightarrow$ Load $\rightarrow$ farm	2/3 ; save phase missing
Action accounting	3 semantic actions ; 8 state checks; 0 off-trace	15 GUI steps ; 3 off-trace
Phase indexing	waits through launch and menu loading; 0 order violations	10 repeated phase visits ; revisits earlier phases after state changes
Cue grounding	4/4 ; app icon, Load button, save identity, hourglass	2/4 ; app icon and Load button only
Efficiency anchors	148.2 s ; 8 shots; 16 calls; 43.3 k tokens	201.1 s ; 15 shots; 17 calls; 43.2 k tokens

Instruction-only replay reproduces the failure mode predicted in Section 3.4. The actor grounded the app icon and the Load button, but exhausted the 15-step budget without selecting the `VLM/VLMBot` save and revisited completed phases ten times after the interface changed. The failure therefore lies in phase indexing and post-action checking rather than low-level GUI actuation.

Demo-guided replay reaches the final farm state with comparable model-call and token totals (16 vs 17 calls, 43.3 k vs 43.2 k tokens). Action ICL grounds all four long-tail cues, including the hourglass save icon and the player-and-farm save label that instruction-only replay misses. Demonstration-conditioned phase indexing keeps the actor on the demonstrated path with zero off-trace actions and zero phase regressions, while WAIT events absorb the asynchronous launch, menu, and save-list transitions explicitly. The 52.9 s wall-time and 46.7% screenshot reductions follow from these two effects rather than from a direct efficiency objective. Each WAIT corresponds to an actor output of kind WAIT in Algorithm 1 (dispatched under the GUI route r_t of the typed action $a_t = (r_t, \alpha_t, \hat{\psi}_t, \chi_t)$ from Section 3.1), and the absence of off-trace actions corresponds to q_t remaining concentrated on the demonstrated phase sequence.

4.3 Study C: Month-Scale Artifact Self-Evolution Probe

Study C evaluates Syll’s document-mediated self-evolution claim as inspectable artifact change. The scenario unfolds as a five-episode time-stamped sequence over a relative month, in which a user teaches project-review preferences and a work-report voice, later returns to reuse them, refines an existing preference, and eventually turns the recurring workflow into a Friday routine. The sequence is executed by a deterministic artifact checker rather than a live human study, and Syll is credited only for changes that appear as before/after document diffs, approval records, reuse hits, or routine artifacts. The full configuration appears in Table 9.

Table 9. Study C protocol card. Time is relative scenario time, written out as week and day labels; it is not a human-subject longitudinal study.

Component	Design choice	What it tests
Task	Project Aurora weekly-review workflow with user preferences, work-report voice, refinement, and routine promotion	Whether user-facing and agent-facing artifacts shape later work
Conditions	Full Syll persists artifacts; Frozen Workspace resets them before the next episode	Whether later behavior depends on durable workspace state
Relative timeline	Week 1 Day 1 preference capture; Week 1 Day 3 style alignment; Week 2 Day 8 reuse; Week 3 Day 15 refinement; Week 5 Day 29 routine promotion	Delayed reuse over a scripted month-scale schedule, without claiming a real longitudinal deployment
Audit artifacts	<code>USER.md</code> , <code>SOUL.md</code> , approvals, routines, before/after/diff files, and ten episode records (five per condition) in <code>e3_results.csv</code> and <code>manifest.json</code>	Public evidence for capture, reuse, refinement, and promotion

We compare two conditions. In **Full Syll**, `USER.md`, `SOUL.md`, and routine artifacts persist across episodes. In **Frozen Workspace**, each episode performs the same per-episode extraction, but the resulting artifacts are reset before the next episode begins. The two conditions therefore differ only in artifact persistence, so any difference in later reuse, refinement, or routine promotion is attributable to it.

Table 10. Study C month-scale artifact co-evolution ledger. Cells use metric style so the artifact transition is visible at a glance.

Time	Episode	Artifact	Full Syll	Frozen Workspace
Week 1 Day 1	Preference capture	<code>USER.md</code>	+5 USER facts ; Aurora source, format, length, delivery preference; diff saved	captures current-turn facts; reset before next episode
Week 1 Day 3	Style alignment	<code>SOUL.md</code>	+4 SOUL rules ; calm technical voice approved; diff saved	accepts current-turn style patch; reset before next episode
Week 2 Day 8	Joint reuse	both	reuse 5/5 + 4/4 ; 0 clarifications; 39 user words saved	0/5 facts + 0/4 rules retained; 1 clarification
Week 3 Day 15	Preference refinement	<code>USER.md</code>	refine=1 ; Blockers made explicit; USER diff saved	missing prior context; 1 clarification; no refinement diff
Week 5 Day 29	Routine promotion	routine	routine=1 ; refined Blockers, voice policy, confirmation retained	no qualified routine; 1 clarification

Both conditions capture preferences and style rules within a single turn, showing that one-shot extraction does not require persistence. The conditions diverge from Week 2 onward, where only Full Syll reuses the captured artifacts in response to the short request “Do my Aurora weekly review.” At Week 3, only Full Syll produces an auditable `USER.md` diff that refines the Blockers preference; Frozen Workspace lacks the prior context and cannot generate a refinement. At Week 4, only Full Syll promotes the recurring workflow into a qualified `aurora_weekly_review` routine that references the Aurora source file, the three-section format, the refined Blockers preference, the `Work-Report Voice` rule, and the draft-first confirmation. The contrast therefore demonstrates that artifact persistence, rather than per-episode model recall, is what enables later reuse, refinement, and routine promotion.

4.4 Discussion

Scope and limitations. Each study has a deliberately narrow scope. Study A uses three manually constructed route probes on one machine, and therefore validates the selected routes rather than adversarial routing coverage. Study B is a single paired clean replay ($n = 1$ per condition), and tests the demonstration schema and replay loop rather than statistical robustness or environmental perturbation such as resolution, theme, or language change. Study C is a deterministic artifact checker with scripted user turns, and verifies workspace and routine behavior rather than model-side extraction quality, population-level preference modeling, or long-term user retention. Larger task sets, perturbation studies, and human audit studies are left for future work.

Reproducibility. The release package includes task setup scripts, Study C’s deterministic driver, success checks, run-date prompt and provider configuration, workspace layout, JSONL event logs, GUI replay evidence, and the pricing table used for cost fields. API keys, channel credentials, and private user data are excluded. Table 11 reports aggregate metadata for Studies A and B, and Table 6 and Table 8 give the per-run breakdowns.

Table 11. Aggregate run metadata for the model-driven studies A and B. Study C is omitted because its driver is deterministic and not a model-efficiency anchor.

Aggregate	Wall time	Calls	Input tokens	Output tokens	Shots
A-suite (3 probes)	42.8 s	10	140.3 k	0.8 k	4
B-pair (2 runs)	349.3 s	33	86.0 k	0.5 k	23

5 Conclusion

This paper presents Syll as an **open-source multimodal agent harness** for **teachable personal automation**. The core insight is that practical computer use is not only a model problem, but also a problem of **routing** and **translation**. An effective system must route work across MCP/API tools, CLI execution, and GUI control, while translating human procedures into reusable skills and translating agent behavior back into reviewable multimodal evidence.

Syll implements this view through a self-hosted modular runtime with persistent local artifacts for memory, skills, routines, traces, approvals, and governance. Our mechanism-oriented studies support this design by validating multimodal routing, teachable GUI replay, and persistent local artifacts. Together, these results suggest that open-source agent systems can become more usable and trustworthy when **teachability**, **auditability**, and **extensibility** are treated as first-class system objectives rather than afterthoughts.

Syll still has clear scope limitations. The current evaluation is designed to validate core mechanisms rather than establish large-scale benchmark superiority across diverse operating systems, interface distributions, or user populations. Important next steps include broader task suites, stronger visual grounding verification, longer-horizon memory consolidation, richer MCP/API integrations, and more robust control policies for side-effectful actions. We hope Syll can serve as a practical foundation for user-owned personal automation systems that people can teach, inspect, and continuously extend.

References

- [1] Reyna Abhyankar, Qi Qi, and Yiyang Zhang. Osworld-human: Benchmarking the efficiency of computer-use agents. *arXiv*, abs/2506.16042, 2025.
- [2] Anthropic. Computer use tool. <https://docs.anthropic.com/en/docs/build-with-claude/computer-use>, 2026. Accessed 2026-04-24.
- [3] Allen Cypher. Eager: Programming repetitive tasks by example. In *Proceedings of the SIGCHI conference on Human factors in computing systems*, pages 33–39, 1991.
- [4] HKUDS. nanobot: An ultra-lightweight personal ai agent framework. <https://github.com/HKUDS/nanobot>, 2026. Accessed 2026-04-29.
- [5] Leslie Pack Kaelbling, Michael L Littman, and Anthony R Cassandra. Planning and acting in partially observable stochastic domains. *Artificial intelligence*, 101(1-2):99–134, 1998.
- [6] Jeonghye Kim, Sojeong Rhee, Minbeom Kim, Dohyung Kim, Sangmook Lee, Youngchul Sung, and Kyomin Jung. Reflect: World-grounded decision making in llm agents via goal-state reflection. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 33421–33453, 2025.
- [7] Thomas Kwa, Ben West, Joel Becker, Amy Deng, Katharyn Garcia, Max Hasin, Sami Jawhar, Megan Kinniment, Nate Rush, Sydney Von Arx, et al. Measuring ai ability to complete long software tasks. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.
- [8] Gilly Leshed, Eben M Haber, Tara Matthews, and Tessa Lau. Coscripter: automating & sharing how-to knowledge in the enterprise. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, pages 1719–1728, 2008.
- [9] Model Context Protocol. What is the model context protocol? <https://modelcontextprotocol.io/docs/getting-started/intro>, 2026. Accessed 2026-04-24.
- [10] OpenAI. Harness engineering: Leveraging codex in an agent-first world. <https://openai.com/index/harness-engineering/>, 2026. Published 2026-02-11, accessed 2026-04-29.
- [11] OpenClaw Contributors. OpenClaw: Open-source self-hosted ai agent framework. <https://github.com/openclaw/openclaw>, 2025. Accessed 2026-04-29.

- [12] Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, et al. Ui-tars: Pioneering automated gui interaction with native agents. *arXiv preprint arXiv:2501.12326*, 2025.
- [13] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems*, 36:68539–68551, 2023.
- [14] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652, 2023.
- [15] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh J Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, et al. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *Advances in Neural Information Processing Systems*, 37:52040–52094, 2024.
- [16] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652, 2024.
- [17] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.
- [18] Tom Yeh, Tsung-Hsiang Chang, and Robert C Miller. Sikuli: using gui screenshots for search and automation. In *Proceedings of the 22nd annual ACM symposium on User interface software and technology*, pages 183–192, 2009.
- [19] Yichun Zhang, Xiangwu Guo, Yauhong Goh, Jessica Hu, Zhiheng Chen, Xin Wang, Difei Gao, and Mike Zheng Shou. Showui-aloha: Human-taught gui agent. *arXiv preprint arXiv:2601.07181*, 2026.
- [20] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, 2024.

Supplementary Material

Contents

A	Author Information	17
B	Additional Implementation Details	17
C	Formal Model	17
C.1	Optimization with Auditability	18
D	Limitations	19
E	Reproducibility Statement	19

A Author Information

We provide author contributions and contact information together in this appendix as follows:

- **Borui Zhang:** Project leader of Syll; coordinated the overall research effort, guided the project direction, and contributed to code development.
Contact: zhang-br21@mails.tsinghua.edu.cn
- **Bo Zhang:** Led the framework design, was primarily responsible for most of the system framework construction, and took the main role in code implementation and maintenance.
Contact: bo-zhang23@mails.tsinghua.edu.cn
- **Chenghao Jiang:** Mainly designed the multimodal GUI grounding and localization tools and contributed to code development.
Contact: chenghao.jiang2022@outlook.com
- **Minglei Shi:** Participated in project discussions, contributed constructive suggestions, and contributed to code and feature development.
Contact: sml25@mails.tsinghua.edu.cn
- **Jie Zhou:** Provided overall guidance and supported the project with resources.
Contact: jzhou@tsinghua.edu.cn
- **Jiwen Lu:** Corresponding author; provided overall guidance and supported the project with resources.
Contact: lujiwen@tsinghua.edu.cn

B Additional Implementation Details

The implementation is partitioned across `syll/agent/` for the core loop, context, skills, GUI planning, and tools; `syll/channels/` and `syll/bus/` for adapters and message flow; `syll/cron/` for scheduled routines; `syll/recorder/` for desktop recording; and `syll/web/` for FastAPI routes and the browser UI.

C Formal Model

Setup and observations. Let $X_t = (X_t^{\text{os}}, X_t^{\text{app}}, X_t^{\text{web}}, X_t^{\text{file}}, X_t^{\text{user}}, G_t)$ be the partially observed computer state and let $A_t = (W_t, H_t, R_t, T_t, V_t, P_t, \Gamma_t)$ be the agent-owned artifact state. The joint state is $S_t = (X_t, A_t)$ and the observable history before the current decision is $h_t = (m_{1:t}, a_{1:t-1}, o_{1:t-1}, z_{1:t-1})$, with belief $b_t(S) = \mathbb{P}(S_t = S \mid h_t)$. Channel events are observations of this state:

$$\xi_t = (c_t, e_t) \sim O(\cdot \mid S_t), \quad m_t = \text{Adapter}_{c_t}(e_t).$$

Process dynamics. The context builder is a lossy prompt projection, not a sufficient statistic:

$$\Phi : \mathcal{A} \times \mathcal{M} \rightarrow \mathcal{C}, \quad C_t = \Phi(A_t, m_t) = (W_t, H_t, R_t, T_t, V_t, P_t).$$

The policy and executor decompose decisions into route choice, concrete action, expected postcondition, confirmation status, observation, and evidence:

$$\begin{aligned} r_t &\sim \pi_{\theta}^R(\cdot \mid C_t), & (\alpha_t, \hat{\psi}_t, \chi_t) &\sim \pi_{\theta}^A(\cdot \mid C_t, r_t), \\ (o_{t+1}, z_{t+1}) &= \text{Exec}_{r_t}(\alpha_t, X_t), & A_{t+1} &= U_A(A_t, m_t, a_t, o_{t+1}, z_{t+1}, \hat{\psi}_t). \end{aligned}$$

Skills as options. A full skill is $s = (\mu_s, \Omega_s, \pi_s, \beta_s, \Psi_s, \Gamma_s)$, an option with metadata, initiation predicate, intra-skill policy, derived termination, postconditions, and evidence schema. A demonstration induces $D_s = \{(I_i, \ell_i, \alpha_i, \psi_i, z_i)\}_{i=1}^n$ and terminates by

$$\beta_s(S_t) = \mathbf{1}[\text{Match}(I_t, \Psi_s^{\text{terminal}})], \quad \Psi_s^{\text{terminal}} = \text{meta.success.check}.$$

Replay maintains $q_t(i) = \mathbb{P}(p_t = i \mid \hat{\tau}_{\leq t}, D_s)$ with monotone kernel $P_{\text{phase}}(i \mid j)$ that assigns $1 - p_{\text{adv}}$ to $i = j$, p_{adv} to $i = j + 1$, and 0 otherwise, retaining boundary mass at the terminal phase. The update and marginal policy are

$$\begin{aligned} q_t(i) &\propto \text{Match}(I_t, \psi_i) \sum_j P_{\text{phase}}(i \mid j) q_{t-1}(j), \\ \pi_s(a \mid C_t, D_s) &= \sum_i q_t(i) [\lambda \delta_{\alpha_i}(a) + (1 - \lambda) \pi_{\theta}(a \mid I_t, \ell_i, \psi_i)]. \end{aligned}$$

Algorithm 1 is the MAP implementation: it stores only $p = \arg \max_i q_t(i)$. Composition is admissible when $\text{Entails}_A(\Psi_{s_1}^{\text{terminal}}, \Omega_{s_2}) = 1$.

ReAct as a degenerate case. The ReAct family interleaves free-form reasoning traces with environment actions [17]; Reflexion and ReFlAct add verbal feedback and goal-state reflection [6, 14]. It is recovered by taking a single channel, collapsing A_t to a textual transcript H_t , removing persistent evidence artifacts and skills, and fixing the route to the environment:

$$(\tau_t, \alpha_t) \sim \pi_{\theta}(\cdot \mid H_t).$$

Thus ReAct is the monolithic-text, no-option, no-audit special case.

C.1 Optimization with Auditability

Auditability is modeled as a constraint rather than a reward bonus:

$$\max_{\pi} \mathbb{E}_{\pi} \left[\sum_{t=0}^T \gamma^t R(S_t, a_t) \right] \quad \text{s.t.} \quad \mathbb{E}_{\pi}[\text{Coverage}(z_{1:T}, \Psi)] \geq \kappa, \quad a_t \in \mathcal{A}_{\text{side}} \Rightarrow \chi_t = 1.$$

The Lagrangian form subtracts $\lambda(\kappa - \mathbb{E}[\text{Coverage}(z_{1:T}, \Psi)])$, while the runtime enforces the constraint directly through confirmations, logs, screenshots, keyframes, and diffs.

Propositions and assumptions. **P1: Markov augmentation.** Given P_X , O , and U_A , the augmented process over $S_t = (X_t, A_t)$ is Markov before the current event is observed: X_{t+1} depends on X_t and a_t , while A_{t+1} depends on A_t and the current message, action, observation, evidence, and expected postcondition. **Assumption 1: artifact-summary ideal.** The artifact updater is designed to preserve history features needed for future action selection; in practice $C_t = \Phi(A_t, m_t)$ is lossy. Under this ideal, the standard belief-MDP reduction applies [5]. **P2: demonstration as option.** D_s induces an option with initiation predicate Ω_s , intra-option policy π_s , and terminal-check termination β_s ; therefore it is a semi-MDP action. **P3: ReAct degeneracy.** Under the degenerate setting above, the typed artifact-option process reduces to ReAct because the only remaining decision is emitting a reasoning trace and an environment action from the textual transcript.

D Limitations

Syll inherits the main limitations of current agent systems. GUI execution can fail because of visual grounding errors, high-DPI coordinate mismatch, hidden state, modal dialogs, permissions, network delays, or application updates. The demonstration loop can reduce repeated effort, but a single recording is not a guarantee of robust generalization across all UI variants.

The current artifact also has evaluation gaps. The public demos show plausible workflows, but the project still needs controlled quantitative results against strong baselines. Human-friendly auditability is a core claim, so it should be measured with user studies rather than inferred from design alone.

Finally, self-hosting gives users control but also shifts responsibility to local configuration. API keys, channel credentials, desktop permissions, filesystem access, and GUI automation privileges must be handled carefully. For safety, destructive file operations, external messaging, purchases, account changes, or other side effects should require explicit confirmation and clear logs.

E Reproducibility Statement

Syll is released as an open-source Python package and repository. The package, CLI, and import path are `syll`. The current codebase uses Python 3.11+ and includes FastAPI web routes, Typer CLI commands, LiteLLM model access, PyQt-based desktop companion support, channel adapters, GUI tools, recorder modules, and tests for web routes, tool validation, GUI planning, voice routes, cron behavior, and dashboard integration.

To reproduce the artifact-level setup:

- install with `python -m pip install syll` or from source with `pip install -e ".[dev]"`;
- run `syll onboard` to create the local configuration and workspace;
- configure the chat/planner/actor model endpoints and any channel credentials in the local config;
- run `syll wake` for the server and web UI, and optionally `syll ghost` for the desktop companion process;
- run `pytest` and `ruff check syll/` for implementation checks.

For future benchmark results, each reported run should include the model versions, provider endpoints, operating system, display resolution, selected screen, GUI actor mode, maximum steps, channel configuration, random seeds where applicable, and full execution traces with screenshots or videos sufficient to audit success and failure cases.